

工厂参数（Factory Parameter）完整链路分析

一、数据模型

参数列表（共14个字段）

序号	字段名	大小(byte)	对应 DID	说明
1	arr_part_num	16	F187	零部件号
2	arr_materias_num	16	F189	物料号（与软件版本号复用）
3	arr_ecu_sn	34	F18C	ECU 序列号
4	arr_vin	17	F190	车身 VIN 码
5	arr_hd_ver	16	F191/F193	硬件版本号
6	arr_sw_ver	16	F195/F1A0	软件版本号
7	arr_repair_shopcode	10	F198	维修店代码（指纹信息）
8	arr_programming_date	4	F199	编程日期
9	arr_installation_date	4	F19D	安装日期
10	arr_systemconfigure1	16	F1A1	系统配置字1（ADAS功能开关）
11	arr_systemconfigure2	16	F1A2	系统配置字2
12	arr_systemconfigure3	16	F1A3	系统配置字3
13	arr_fawreserved	16	F1A7	一汽预留
14	arr_conf_form	1	F1A9	远程配置字

双重存储

MCU 侧和 SOC 侧各自独立存储一份工厂参数，两套存储不存在主从同步关系：

	MCU 侧	SOC 侧
存储框架	PM (Parameter Manager)	Nvm (NVRAM Manager)
数据结构	pm_factory_param_block_t 结构体	16 个独立的 g_xxx[] 全局数组
底层介质	MCU 内部 Flash (Nvm)	SOC 侧 Flash (Nvm + Fls)
定义文件	FactoryParam_Type.h.35-51	FactoryParam_Cfg.c.14-28
注册方式	DECLLEAR_PARAM_BLOCK_EXT(pm_factory, CFG_FACTORY, ...)	Nvm Block Descriptor（由 MICROSAR 配置工具生成）

二、五条读写路径总览

	5A8650 芯片
	UDS DID handler
① CAN 诊断仪	→ MCU (安全岛)
	└ read_param_by_id() → PM → Nvm
	└ write_param_by_id() → PM → Nvm
② DoIP 诊断仪	→ SOC (应用处理器) → Nvm + Fls
	└ UDS DID handler
	└ Nvm_WriteBlock() + Fls.Sync()
③ 跨域读请求(SOC-MCU)	SOC → DCMS topic → MCU
	└ read_param_block_by_id() → PM
④ MCU Shell 命令	MCU shell → DCMS topic → SOC
	└ memcpy g_xxx → Nvm_WriteAll()
⑤ 系统属性同步	SOC: rw.oem.sn, /rw.oem.vin
	后台线程 1s 周期检查 + 标志位触发

三、路径① — UDS CAN 诊断（MCU 侧，通过 PM）

入口：诊断仪通过 CAN 总线发送 UDS 请求（22=ReadDataByIdentifier, 2E=WriteDataByIdentifier）

存储层：MCU PM → Nvm

读流程

以 F187（零部件号）为例（FactoryParam.c.463-483）：

```
CAN 22 F187
→ FactoryParam_FAW_Part_Number.16Bytes F187_ReadData()
→ read_param_by_id(CFG_FACTORY, &PartNumTemp, pm_factory_param_block_t, arr_part_num)
→ PM 层从 Nvm 读取 pm_factory_param_block_t.arr_part_num
→ memcpy(Data, PartNumTemp, 16)
→ 返回 DCM_E_OK
```

MCU 侧实现了的 DID 读函数：F187, F189, F18C, F190, F191, F193, F195, F198, F199, F19D, F1A0, F1A1, F1A2, F1A3, F1A7, F1A9 —— 全部使用 read_param_by_id(CFG_FACTORY, ...) 从 PM 读取。

写流程

以 F190（VIN 写）为例（FactoryParam.c.1003-1023）：

```
CAN 2E F190 [17 bytes VIN]
→ CAN_Write(2E, F190, 17, VIN)
→ Nvm_Write(F190, VIN, 17)
→ F190 写入成功
```

```
→ FactoryParam_FAW_Vehicle_Identification_Number_17Bytes_F190_WriteData()
→ write_param_by_id(CFG_FACTORY, &VehIdNumTemp[0], pm_factory_param_block_t, arr_vin)
→ PM 层写入 NvM
→ 返回 DCM_E_OK
```

MCU 侧实现了的 DID 写函数：F190, F1A1, F1A2, F1A3, F1A7, F1A9, F198, F199, F19D —— 全部使用 write_param_by_id(CFG_FACTORY, ...) 写入 PM

四、路径② — UDS DoIP 诊断（SOC 侧，通过 NvM）

入口：诊断仪通过以太网（DoIP）发送 UDS 请求

存储层：SOC NvM → Flash（通过 Fls_Sync()）

读流程

以 F18C（ECU序列号）为例（FactoryParam.c:427-435）：

```
DoIP 22 F18C
→ DataServices_0xF18C_ECU_Serial_Number_34Bytes_ReadData()
→ memcpy(Data, g_ecu_serial_number, 34) // 直接拷贝 NvM 全局变量
→ 返回 E_OK
```

SOC 侧 DID 读：全部通过 memcpy(Data, g_xxx, sizeof(g_xxx)) 直接读 NvM 全局变量，无需跨域请求 MCU。

特殊 — 软件版本号读（F189/F195/F1A0）（FactoryParam.c:364-418）：

```
DoIP 22 F189/F195/F1A0
→ FactoryParam_Read_SoftVersion()
├─ DCM_INITIAL: 返回 DCM_E_PENDING, 启动异步查询
├─ DCM_PENDING:
│  ├─ 如果已收到 → memcpy(Data, Software_Version, 16)
│  ├─ 如果没收到且未超时 → 每1秒调用 Software_Version_Get()
│  │  └─ dcms_mcu_topic_send(DCMS_TOPIC_GET_SOFTWARE_VERSION, ...) // 向 MCU 查询!
│  │  └─ MCU 返回版本号 → Read_SoftWare_Version_Callback() 填充 Software_Version[12..15]
│  └─ 超时 → DCM_E_NOT_OK
└─ 其他: DCM_E_NOT_OK
```

软件版本号是唯一一个 SOC 需要向 MCU 实时查询的参数，因为 OTA 升级后版本号会变化。版本号组成：

- Software_Version[0..7]：硬编码 "3629801"
- Software_Version[8..11]：车型代码（如 HX=E009, QX=P301）由 SoftVersion_CarModel_Convert() 根据车型 ID 动态填充
- Software_Version[12..15]：从 MCU 实时查询的版本号

写流程

以 F190（VIN 写）为例（FactoryParam.c:462-473）：

```
DoIP 2E F190 [17 bytes VIN]
→ DataServices_0xF190_FAW_Vehicle_Identification_Number_17Bytes_WriteData()
→ FactoryParam_Write_ProductParam(0xF190, NvMBlock_VIN, g_vin, ...)
├─ DCM_INITIAL:
│  ├─ NvM_WriteBlock(NvMBlock_VIN, &Data[0]) // 发起 Flash 写
│  └─ 返回 DCM_E_PENDING (等待 Flash 完成)
├─ DCM_PENDING:
│  ├─ 如果上次失败 → 重试 NvM_WriteBlock()
│  ├─ NvM_GetErrorStatus() + Fls_Sync()
│  ├─ 如果完成 → memcpy(g_vin, Data, 17), 返回 E_OK
│  └─ 如果超时 (>1200次) → 返回 E_NOT_OK
└─ 其他: E_NOT_OK
→ 如果成功 → FactoryParam_DcmVinSyncFlagSet() // 触发系统属性同步
```

关键点：SOC 写操作不同步到 MCU PM。只写 SOC NvM。

特殊 — F1A1（系统配置字1）写（FactoryParam.c:699-766）：

F1A1 写入时除了存 NvM，还会解析 30+ 个 ADAS bit 标志位，通过 DCMS topic 发送给行车/FL 模块：

```
DoIP 2E F1A1
→ 解析 Data[0..6] 的 bit 位 → driving_param_config / fl_param_config
→ dcms_mcu_topic_send(DCMS_TOPIC_DMM_PARAM_CONFIG, ...) // 发送给行车模块
→ dcms_mcu_topic_send(DCMS_TOPIC_DMM_PARAM_CONFIG_FL, ...) // 发送给 FL 模块
→ FactoryParam_Write_ProductParam(...) // 正常 NvM 存储
```

五、路径③ — DCMS 跨域读（SOC → MCU PM）

入口：SOC 侧需要读取 MCU PM 中存储的某个工厂参数

协议：Client-Server 模式，SOC 为 Client，MCU 为 Server

MCU 侧（Server）注册

FactoryParam.c:327-338：

```
void Factory_Params_Sync_Init(void) {
    // MCU 侧注册两个 server callback
    dcms_mcu_topic_setup_callback(DCMS_TOPIC_FACTORY_PARAM_SYNC,
        Dcms_Mcu_Service_Factory_Param_Read_Callback, ...); // 读服务
    dcms_mcu_topic_setup_callback(DCMS_TOPIC_WRITE_FACTORY_PARAM_SYNC,
        Dcms_Mcu_Service_Factory_Param_Write_Callback, ...); // 写服务
}
```

Topic 配置（MCU: SERVER=1）：

Topic ID	Topic 字符串
DCMS_TOPIC_FACTORY_PARAM_SYNC	/sys/mcu_param_manager/read/v2
DCMS_TOPIC_WRITE_FACTORY_PARAM_SYNC	/sys/mcu_param_manager/write/v2

跨域读请求

FactoryParam.c:22-176:

```
SOC Client → dcms_mcu_topic_send("/sys/mcu_param_manager/read/v2", {item name: "arr_vin"})
└─ MCU Server 收到
    └─ read_param_block_by_id(CFG_FACTORY, &pm_factory_param_list, ...) // 读全部参数到内存
        └─ strcmp(item_name, "arr_vin") → 匹配, 取出 pm_factory_param_list.arr_vin
        └─ dcms_mcu_service_server_set_ackdata(..., respond) // 返回 McuParamManagerRdRsp
    └─ SOC Client 收到响应: {result, len, data[200]}
```

支持读单个参数 (如 "arr_vin") 或全部参数 ("all")。

当前 SOC 侧使用情况: security_cert_manager 模块通过 /sys/mcu_param_manager/read/v1 (旧版本 v1) 读取工厂参数 (security_cert_manager.h:56)

跨域写请求

FactoryParam.c:179-320:

```
SOC Client → dcms_mcu_topic_send("/sys/mcu_param_manager/write/v2", {item_name, len, data})
└─ MCU Server 收到
    └─ FactoryParam_Write0emArgu(item_name, data)
        └─ strcmp key → write_param_by_id(CFG_FACTORY, data, pm_factory_param_block_t, arr_xxx)
    └─ 返回 {result: 0=成功, 1=失败}
```

六、路径④ — ECU Shell 调试 (MCU Shell → SOC NvM)

纯调试通道, MCU 侧由 #ifdef, MC_SHELL_SUPPORT 宏控制。

Topic 定义

Topic ID	Topic 字符串	方向	MCU	SOC
DCMS_FACTORY_PARAM_SET	/sys/product_param_set/v1	MCU → SOC	PUB=1	SUB=1
DCMS_FACTORY_PARAM_SET_RESULT	/sys/product_param_set_result/v1	SOC → MCU	SUB=1	PUB=1

写流程

```
MCU Shell 命令
┌─
└─ factory_param_nv_set [param_id] [str1] [str2]...
    └─ 参数: param_id=1~15, 后面跟 ASCII 字符串 (总长度必须等于参数长度)
    └─ 例: factory_param_nv_set 5 LSVXXXXXXXX00000000 (写入17字节VIN)
┌─
└─ hexfactory_param_nv_set [param_id] [base_addr] [byte0] [byte1]...
    └─ 参数: 十六进制逐字节, 支持分次写入大参数 (每批最多128字节)
    └─ 例: hexfactory_param_nv_set 4 0 0x31 0x32 0x33 ...
┌─
└─
    ▼ MCU 侧 [Diag_CrossEcuShell.c:120-208]
构造 msg_factory_param_request_t {param_id, param_data[128], param_len}
└─ dcms_mcu_topic_send_msg(DCMS_FACTORY_PARAM_SET, ...)
    └─ IPC 跨域 ───────────────────
    ▼ SOC 侧 [cross_ecu_shell.c:73-140]
cross_factory_param_set_callback()
┌─ 校验: data!=NULL, len匹配, param_id范围[1,15], param_len匹配
└─ memcpy(&g_xxx[0], param_data, param_len) // 直接写 NvM 全局变量
└─ param_id==4(SN) → FactoryParam_0emSnSyncFlagSet()
└─ param_id==5(VIN) → FactoryParam_0emVinSyncFlagSet()
└─ NvM_WriteAll() + Fls_Sync() // 落盘
└─ cross_factory_param_set_response()
    └─ dcms_mcu_topic_send_msg(DCMS_FACTORY_PARAM_SET_RESULT, result_string)
    └─ IPC 跨域 ───────────────────
    ▼ MCU 侧 [Diag_CrossEcuShell.c:69-84]
factory_param_nv_set_callback()
└─ 缓存结果到 factory_param_respond_info[]
┌─
└─
    ▼ 用户查询结果
factory_param_nv_get → shell_print(result)
```

初始化:

- MCU: Diag_CrossEcuShell.c:91-101 — 注册 SUB callback for DCMS_FACTORY_PARAM_SET_RESULT
- SOC: cross_ecu_shell.c:148-158 — 延迟 2500ms 后注册 SUB callback for DCMS_FACTORY_PARAM_SET

两张参数表的关键区别

SOC 侧 factory_param_info[] .param_addr 指向真正的 NvM 全局变量 (cross_ecu_shell.c:27-46), 例如 &g_part_number[0], 所以 memcpy(param_addr, ...) 就是直接写 NvM。
MCU 侧 factory_param_info[] .param_addr 指向本地 static 临时缓冲区 (Diag_CrossEcuShell.c:43-61), 例如 &g_part_number1[0] (注意后缀 1), 仅用于 shell 命令中构造消息。

七、路径⑤ — 系统属性同步 & DSSAD 上报 (SOC 内部)

系统属性同步

FactoryParam.c:1209-1313 — 后台线程 FactoryParam_SyncThread, 1s 周期循环:

```
t=10s: FactoryParam_System_Configuration1_topic_send()
└─ 解析 g_sys_configure_1[] bit 位 → 发送 DMM/FL topic
t=20s: 读系统属性 rw.oem.sn / rw.oem.vin → 打印对比
t=21s: 如果 SN 不一致 → dji_mini_param_set_system_property("rw.oem.sn", ...)
t=22s: 如果 VIN 不一致 → dji_mini_param_set_system_property("rw.oem.vin", ...)
标志位触发: s0emSnSyncFlag/s0emVinSyncFlag=true → 立即同步
```

触发标志位的场景：

- DoIP 2E F190 与 VIN 成功 → [FactoryParam.c:470](#)
- ECU Shell 写 SN → [cross_ecu_shell.c:113](#)
- ECU Shell 写 VIN → [cross_ecu_shell.c:118](#)

DSSAD 周期上报

FactoryParam.c:62-96 — 每 60s（代码中 30s 但注释说明实际约 60s 周期）：

```
FactoryParam_SendtoDssad()  
→ 组装 Dssad_Factory_Param_t {VIN, HW_Ver, SN, SW_Ver}  
→ dcms_mcu_topic_send(DCMS_FAW_DSSAD_FACTORY_PARAM,...) // "/dssad/faw_factory_param.v1"
```

下电打印

FactoryParam.c:126-160 — `FactoryParam_NvmBlockPrint()` 在下电时打印所有 NvM block 数据和错误状态。

八、五条路径对比总结

维度	① UDS CAN	② UDS DoIP	③ DCMS 跨域	④ ECU Shell	⑤ 系统属性同步
触发方	CAN 诊断仪	DoIP 诊断仪	SOC 应用代码	MCU Shell 命令	SOC 后台线程
物理链路	CAN 总线	车载以太网	芯片内部 IPC	芯片内部 IPC	SOC 内部
写入目标	MCU PM → NvM	SOC NvM	MCU PM → NvM	SOC NvM	QNX 系统属性
API	<code>read/write_param_by_id</code>	<code>NvM_WriteBlock + Fls_Sync</code>	<code>dcms_mcu_topic_send (C/S)</code>	<code>dcms_mcu_topic_send_msg (Pub/Sub)</code>	<code>dji_mini_param_get/set</code>
F1A1 ADAS解析	无	有 (bit解析 → DMM topic)	无	无	无 (仅10s发送DMM)
写后同步MCU	N/A (本就地)	不同步	N/A (本就地)	不同步	N/A
使用场景	生产制造/售后	生产制造/售后	安全认证等模块	开发调试	系统属性消费者
代码开关	无	无	无	<code>#ifdef MC_SHELL_SUPPORT</code>	无

核心设计原则：MCU 和 SOC 的工厂参数存储是**完全独立的两套**，分别对应 CAN 诊断和 DoIP 诊断两条物理通道。生产制造时，工厂工会**同时通过两条通道写入相同的数据**（CAN 写 MCU，DoIP 写 SOC），保证两侧数据一致。ECU Shell 是纯调试后门，不走诊断协议栈。

我还是不理解，你那0xf187为例分析下doip侧，docan侧如何写入读取，调用关系是怎样的，是否走了跨域请求

- Thought for 56s >
- 好的，让我以 **F187（零部件号）** 和 **F190（VIN码）** 两个具体 DID 为例，逐层追踪调用链。
- Bash** Search for F187 references in MCU FactoryParam.c

```
IN grep -rn "F187\|0xF187" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/FactoryParam/FactoryParam.c 2>/dev/null  
  
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/FactoryParam/FactoryParam.c:459: * @brief DID F187 Read 远程配置字 16byte 可读  
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/FactoryParam/FactoryParam.c:463: FUNC(STD_ReturnType, BootProcess_CODE) FactoryParam_FAW_Part_N  
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/FactoryParam/FactoryParam.c:473: DEBUG_INFO("<factory_param>get F187 arr_conf_form: %x\
```

- Bash** Search for F187 references in SOC FactoryParam.c

```
IN grep -rn "F187\|0xF187\|g_part_number\|HardAndPart" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/FactoryParam/FactoryParam.c 2>/dev/null  
  
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/FactoryParam/FactoryParam.c:132: FactoryParam_NvmDataPrint("part_number ", &g_part_number[  
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/FactoryParam/FactoryParam.c:801: * DiagDid 0xF187_FAW_Part_Number 16Bytes ReadData()  
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/FactoryParam/FactoryParam.c:803: /* \brief 22F187 Reads the FAW_Part_Number.
```

- Thought for 10s >
- Bash** Search DID routing config for F187 in MCU DcmApp

```
IN grep -rn "F187\|0xF187" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/DcmApp/ --include="*.c" --include="*.h" 2>/dev/null | head -20  
  
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/DcmApp/dji_rte.c:56: DID 0xF187_ReadData Svc = FactoryParam_FAW_  
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/DcmApp/DiagSwc_Callback.h:15:// Function: DiagDid 0xF187_FAW_Part_Number 16Bytes ReadData()  
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/oem_feature/diagnosis/DcmApp/DiagSwc_Callback.h:16:extern FUNC(STD_ReturnType, DCM_CALLOUT_CODE) DiagDid_0xF187_FAW
```

- Bash** Search DID routing config for F187 in SOC autosar_adapter

```
IN grep -rn "F187\|0xF187" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/ --include="*.c" --include="*.h" 2>/dev/null | head -20  
  
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/dji_rte.c:82: DID 0xF187_ReadData Svc  
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/Appl/Include/DiagSwc_Callback.h:14:// Function: DiagDid 0xF187_F  
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/Appl/Include/DiagSwc_Callback.h:15:extern FUNC(STD_ReturnType, D
```

- Thought for 3s >
- Read dji_rte.c**
- Thought for 2s >
- Read dji_rte.c**
- Thought for 4s >
- 现在完全清晰了。下面以 F187（零部件号）和 F190（VIN码）为例，逐层追踪。

一、F187 零部件号（只读DID，无写入）

DoIP 侧（SOC）调用链

```
诊断仪 —DoIP—> SA8650 SOC
|
▼
MICROSAR Dcm 模块 (Dcm_Lcfg.c)
| 路由表匹配 DID=0xF187
|
▼
DiagSwc_Callback.c:7 DiagDid_0xF187_FAW_Part_Number_16Bytes_ReadData()
| 从函数指针表取值并调用:
| return rte_config.DID_Read_SvcRsp.DID_0xF187_ReadData_Svc(0pStatus, Data);
|
▼
dji_rte.c:82 函数指针绑定了:
| .DID_0xF187_ReadData_Svc = DataServices_0xF187_FAW_Part_Number_16Bytes_ReadData
|
▼
FactoryParam.c:807 DataServices_0xF187_FAW_Part_Number_16Bytes_ReadData()
| memcpy(Data, HardAndPart_Version_Info, sizeof(g_part_number));
| // HardAndPart_Version_Info 是 ROM 常量: {'3','6','2','9','1','0','0','-','H','X','0','3','0','0','0','0'}
| 然后根据 Diag_GetCarModelValue() 覆盖 Data[8..15]
| 例: E009 -> Data[8]='H', Data[9]='X'
|
|
└ 返回 E_OK -> DoIP 响应回诊断仪
```

关键点: SOC DoIP 读 F187 的数据来源是 ROM 常量 `HardAndPart_Version_Info` + 车型 ID, 不走 NVM, 不走跨域请求。

CAN 侧（MCU）调用链

```
诊断仪 —CAN—> SA8650 MCU (安全岛)
|
▼
AUTOSAR Dcm 模块
| 路由表匹配 DID=0xF187
|
▼
DiagSwc_Callback.c:7 DiagDid_0xF187_FAW_Part_Number_16Bytes_ReadData()
| return rte_config.DID_Read_SvcRsp.DID_0xF187_ReadData_Svc(0pStatus, Data);
|
▼
dji_rte.c:56 函数指针绑定了:
| .DID_0xF187_ReadData_Svc = FactoryParam_FAW_Part_Number_16Bytes_F187_ReadData
|
▼
FactoryParam.c:463 FactoryParam_FAW_Part_Number_16Bytes_F187_ReadData()
| read_param_by_id(CFG_FACTORY, &PartNumTemp, pm_factory_param_block_t, arr_part_num)
|   └ PM 层从 MCU 本地 Nvm 读取 pm_factory_param_block_t.arr_part_num[16]
| memcpy(Data, PartNumTemp, 16)
|
|
└ 返回 DCM_E_OK -> CAN 响应回诊断仪
```

关键点: MCU CAN 读 F187 直接从 MCU 本地 PM/NVM 读取, 不走跨域请求。

F187 没有写入

两边代码里都没有 F187 的 WriteData 函数。F187 是只读 DID, 零部件号由编译期固化和 EOL 工位一次性编程确定。

二、F190 VIN码（可读可写 DID）

DoIP 侧（SOC）读

```
诊断仪 —DoIP—> SOC
|
▼ ... (同上路由机制, 略)
|
▼
dji_rte.c:73 .DID_0xF190_ReadData_Svc = DataServices_0xF190_FAW_Vehicle_Identification_Number_17Bytes_ReadData
|
▼
FactoryParam.c:445 DataServices_0xF190_FAW_Vehicle_Identification_Number_17Bytes_ReadData()
| memcpy(Data, g_vin, sizeof(g_vin)); // 直接读 SOC NVM 全局变量 g_vin[17]
|
|
└ 返回 E_OK
```

DoIP 侧（SOC）写

```
诊断仪 —DoIP—> SOC 2E F190 [17 bytes VIN数据]
|
|
▼ ... (路由)
|
▼
dji_rte.c:42 .DID_0xF190_WriteData_Svc = DataServices_0xF190_FAW_Vehicle_Identification_Number_17Bytes_WriteData
|
▼
FactoryParam.c:462 DataServices_0xF190_FAW_Vehicle_Identification_Number_17Bytes_WriteData()
|
| └ FactoryParam_Write_ProductParam(0xF190, NvMBlock_VIN, g_vin, sizeof(g_vin), Data, ...)
|   └ DCM_INITIAL: Nvm_WriteBlock(NvMBlock_VIN, &Data[0]) // 启动 Flash 写
|     └ 返回 DCM_E_PENDING
|   └ DCM_PENDING: Nvm_GetErrorStatus() + F190_Sync() // 等待 Flash 完成
|     └ 完成后: memcpy(g_vin, Data, 17)
|     └ 返回 DCM_E_OK
|   └ 超时: 返回 E_NOT_OK
|
|
└ if (ret == E_OK):
    FactoryParam_OemVinSyncFlagSet(); // 设标志位, 后台线程稍后同步到 rw.oem.vin
```

CAN 侧（MCU）读

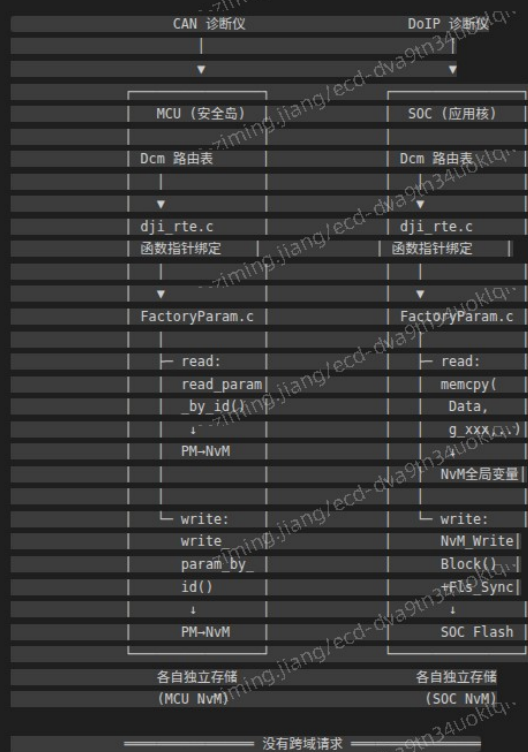
CAN 侧 (MCU) 读

```
诊断仪 —CAN—> MCU
|
| ... (路由)
|
dji_rte.c:59 .DID_0xF190_ReadData_Svc = FactoryParam_FAW_Vehicle_Identification_Number_17Bytes_F190_ReadData
|
FactoryParam.c:435 FactoryParam_FAW_Vehicle_Identification_Number_17Bytes_F190_ReadData()
|   read_param_by_id(CFG_FACTORY, &VehIdNumTemp, pm_factory_param_block_t, arr_vin)
|   |   PM 层从 MCU 本地 NVM 读取 pm_factory_param_block_t.arr_vin[17]
|   |   memcpy(Data, VehIdNumTemp, 17)
|   |
|   |
|   | 返回 DCM_E_OK
```

CAN 侧 (MCU) 写

```
诊断仪 —CAN—> MCU 2E F190 [17 bytes VIN数据]
|
| ... (路由)
|
dji_rte.c:19 .DID_0xF190_WriteData_Svc = FactoryParam_FAW_Vehicle_Identification_Number_17Bytes_F190_WriteData
|
FactoryParam.c:1003 FactoryParam_FAW_Vehicle_Identification_Number_17Bytes_F190_WriteData()
|   write_param_by_id(CFG_FACTORY, &VehIdNumTemp, pm_factory_param_block_t, arr_vin)
|   |   PM 层写入 MCU 本地 NVM
|   |
|   |
|   | 返回 DCM_E_OK
```

三、结论



核心结论:

- 1. UDS DID 读写路径不存在跨域请求。CAN 诊断仪写 MCU PM/NvM, DoIP 诊断仪写 SOC NvM, 两边各自操作本地存储, 互不通知。
- 2. 两侧数据一致性靠生产工艺保证: EOL (End of Line) 工位同时对 CAN 和 DoIP 两条通道写入相同的参数值, 所以出厂时两侧数据一致。
- 3. 唯一的跨域查询例外是软件版本号 (F189/F195/F1A0): SOC DoIP 读到这些 DID 时, 通过 DCMS_TOPIC_GET_SOFTWARE_VERSION → MCU 实时查询版本号, 因为 OTA 升级后版本号会变。但这也只是查询而非写入同步。
- 4. 两套路由表独立但结构镜像: MCU 侧 dji_rte.c:56 和 SOC 侧 dji_rte.c:82 分别把同一个 DID (如 0xF187) 绑定到各自的处理函数, 但处理函数访问的是各自本地的存储 (MCU → PM, SOC → NvM全局变量)。

你主要帮我分析下哪些工厂参数是跨域的, 分析下时序关系, 另外读工厂参数时直接读的内存变量, 不应该是读存储变量吗

Thought for 24s >

好的, 两个问题都很关键。让我逐一深入分析。

Bash

Search MCU side for cross-domain topic receivers of factory params

```
IN grep -rn "DMM_PARAM_CONFIG|dssad/faw_factory_param|GET_SOFTWARE_VERSION|mcu_param_manager/read|mcu_param_manager/write" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/prod

OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_p301/config/app_config/fawhq_p301_dcms_config.h:528: {0, DCMS_TOPIC_FACTORY_PARAM_SYNC, "/sys/mcu_para
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_p301/config/app_config/fawhq_p301_dcms_config.h:529: {0, DCMS_TOPIC_WRITE_FACTORY_PARAM_SYNC, "/sys/mcu_para
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_fw/product/faw/fawhq_e001_10/config/app_config/fawhq_e001_10_dcms_config.h:530: {0, DCMS_TOPIC_FACTORY_PARAM_SYNC, "/sys/mc
```

Bash

Search SOC side for cross-domain factory param topic usage outside main files


```
IN grep -rn "GATEWAY_FACTORY_PARAM\|mcu_param_manager\|FACTORY_PARAM" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/ --include="*.c" --include="*.h" 2>/dev/null | g
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/include/dcms_mcu_config_uds.h:42: DCMS_FACTORY_PARAM_SET,
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/include/dcms_mcu_config_uds.h:43: DCMS_FACTORY_PARAM_SET_RESULT,
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/include/dcms_mcu_config_uds.h:64: DCMS_FAW_DSSAD_FACTORY_PARAM,
```

Thought for 54s >

Bash Check NvM block configuration for RAM mirror pattern

```
IN grep -rn "g_vin\|g_part_number\|g_ecu_serial_number\|g_ecu_hd_version\|NvMBlock_VIN\|NvMBlock_PartNumber\|permanent\|PERMANENT\|NvM_WriteBlock.*g_vin\|NvM_ReadBlock.*g_vin" /home/c-ziming.jiang/
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/Appl/GenData/NvM_Cfg.c:107:static VAR(uint8, NVM_PRIVATE_DATA) N
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/Appl/GenData/NvM_Cfg.c:366: (NvM_RamAddressType)&g_part_n
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/Appl/GenData/NvM_Cfg.c:442: (NvM_RamAddressType)&g_ecu_se
```

Bash Search NvM config for VIN ram block definition

```
IN grep -rn "NvM_ReadBlock\|NvM_WriteBlock\|NvM_RamBlock\|ram_block\|RamBlock" /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_co
OUT /home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/Appl/GenData/NvM_Cfg.c:366: (NvM_RamAddressType)&g_part_n
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/Appl/GenData/NvM_Cfg.c:442: (NvM_RamAddressType)&g_ecu_se
/home/c-ziming.jiang/dsar_app_fw_platform_vip/dsar-hq/src/dsar_app/product/faw/oem_feature/autosar_adapter/microsar_config_uds_v4/Appl/GenData/NvM_Cfg.c:480: (NvM_RamAddressType)&g_vin /*
```

Thought for 17s >

Read NvM_Cfg.c (lines 471-510)

Thought for 7s >

Update Todos

🔖 输出跨域分析和RAM镜像解答

Thought for 1s >

一、哪些工厂参数走了跨域

并非所有工厂参数都跨域。只有以下场景涉及 SOC ↔ MCU 的跨域通信：

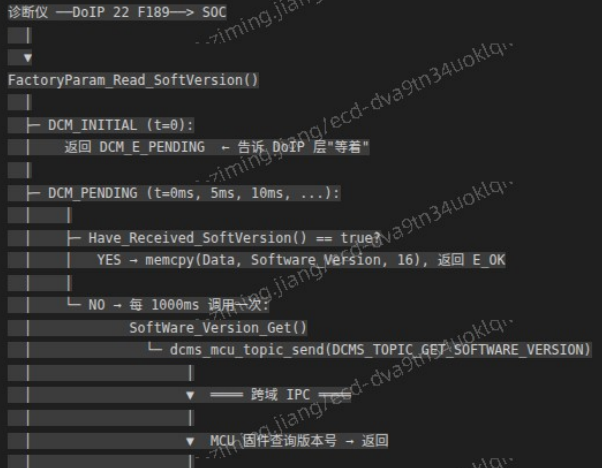
跨域路径汇总

跨域 Topic 总结		
Topic	方向	用途
/sys/mcu_param_manager/read/v2 (DCMS_TOPIC_FACTORY_PARAM_SYNC)	SOC→MCU C/S模式	SOC 读取 MCU PM 中的 任意工厂参数（按名称）
/sys/mcu_param_manager/write/v2 (WRITE_FACTORY_PARAM_SYNC)	SOC→MCU C/S模式	SOC 写入 MCU PM 中的 任意工厂参数（按名称）
/sys/update/app_install/ firmware_version_query	SOC→MCU Pub/Sub	SOC 查询 MCU 软件版本号 → 填充 Software_Version
/sys/product_param_set/v1 (DCMS_FACTORY_PARAM_SET)	MCU→SOC Pub/Sub	MCU Shell 写入 SOC NvM (ECU Shell 调试)
/sys/product_param_set_result/v1 (DCMS_FACTORY_PARAM_SET_RESULT)	SOC→MCU Pub/Sub	SOC 返回写入结果给 MCU (ECU Shell 调试)

各场景详细时序

场景1: DoIP 读软件版本号 (F189/F195/F1A0)

这是 UDS DID 读路径中唯一需要跨域的场景。



```
|
|      └── 跨域 IPC ──
|
|      |
|      Read_Software_Version_Callback()
|      |─ Software_Version[12..15] = resp_msg->version[0..3]
|      |─ Software_Version[8..11] = 车型代码 (SoftVersion_CarModel_Convert)
|      |─ Received_Sw_Version = true
|
|
|      └─ 超时 (>10000ms): 返回 DCM_E_NOT_OK
```

时序关键点: SOC 不阻塞等待, 而是每次返回 DCM_E_PENDING, 让 AUTOSAR Dcm 模块轮询 (约 5ms 一次)。SOC 内部每 1 秒向 MCU 发一次查询, 最多等 10 秒。

场景2: security_cert_manager 读取 MCU 工厂参数

这不是 UDS DID 路径, 而是 SOC 内部模块主动查询 MCU PM:

```
SOC_security_cert_manager 初始化
|
|
|      └─
|
|      dcms_mcu_topic_send("/sys/mcu_param_manager/read/v1", {item_name: "arr_vin"})
|      └── Client-Server 跨域 ──
|
|      └─
|
|      MCU: Dcms_Mcu_Service_Factory_Param_Read_Callback()
|      | read_param_block_by_id(CFG_FACTORY, &pm_factory_param_list, ...)
|      | strcmp(item_name) → 匹配 → memcpy 对应字段
|      | dcms_mcu_service_server_set_ackdata(...)
|      | └── Server-Client 跨域 ──
|
|      └─
|
|      SOC 收到 McuParamManagerRdRsp {result, len, data[200]}
```

场景3: ECU Shell 写 SOC NvM (调试)

```
MCU Shell 输入: factory_param_nvmm_set 5 LSVXXXXXXXX00000000
|
|
|      └─
|
|      MCU: do_factory_param_nvmm_set()
|      | 构造 msg_factory_param_request t {param_id=5, data="LSV...", len=17}
|      | dcms_mcu_topic_send(DCMS_FACTORY_PARAM_SET, ...)
|      | └── Pub-Sub 跨域 ──
|
|      └─
|
|      SOC: cross_factory_param_set_callback()
|      | memcpy(&g_vin[0], param_data, 17) // 写 RAM 镜像
|      | NvM_WriteAll() // 触发全量 Flash 写
|      | Fls_Sync() // 等待 Flash 完成
|      | dcms_mcu_topic_send_msg(DCMS_FACTORY_PARAM_SET_RESULT, "write result...")
|      | └── Pub-Sub 跨域 ──
|
|      └─
|
|      MCU: factory_param_nvmm_set_callback()
|      | 缓存结果到 factory_param_respond_info[]
|
|      └─
|
|      MCU Shell 输入: factory_param_nvmm_get
|      └─ 打印写入结果
```

二、为什么读工厂参数是直接读内存变量而不是读存储

你的怀疑是对的, 直觉上读数据应该从存储 (Flash/NvM) 读。但实际上 memcpy(Data, g_vin, sizeof(g_vin)) 就是读的“存储”——因为 g_vin 就是 NvM 的 RAM 镜像 (RAM Mirror)。

NvM RAM 镜像机制

这是 AUTOSAR NvM 的标准设计模式。查看 NvM 配置 (NvM_Cfg.c.479-480):

```
{ /* NvMBlock_VIN */
  (NvM_RamAddressType)&g_vin /* ← NvMRamBlockDataAddress: g_vin 就是本 Block 的 RAM 镜像 */
  , ...
  NvMBlock_VIN_Crc /* ← CRC 校验缓冲区 */
  , ...
  17u /* NvMNvBlockLength = 17 */
  , ...
  NVM_SELECT_BLOCK_FOR_READALL_ON /* ← 启动时从 Flash 读到 RAM */
  NVM_SELECT_BLOCK_FOR_WRITEALL_ON /* ← 下电时从 RAM 写到 Flash */
}
```

g_vin[17] 不是普通的全局变量, 它是 NvM 配置表中注册的 RAM 镜像地址。NvM 框架保证:

上电启动:
NvM_ReadAll() → 从 Flash 读取 NvMBlock_VIN 的数据 → 写入 g_vin[17] (CRC 校验)

运行时读:
memcpy(Data, g_vin, 17) ← 直接读 RAM 镜像, 始终是最新值, 无需再读 Flash

运行时写:
NvM_WriteBlock(NvMBlock_VIN, &data[0])
└─ 立即写入 g_vin (RAM 镜像)
└─ 调度异步 Flash 写入任务

下电时:
NvM_WriteAll() → 将所有 RAM 镜像 (含 g_vin) 写入 Flash → Fls_Sync()

所以 memcpy(Data, g_vin, 17) 等价于“读存储”——g_vin 在上电时已从 Flash 加载, 在运行中被 NvM 框架维护为与 Flash 一致的权威副本。如果再调用 NvM_ReadBlock(), 本质上也是从同一个 RAM 镜像拷贝一份出来, 是多余的。

MCU 侧同理

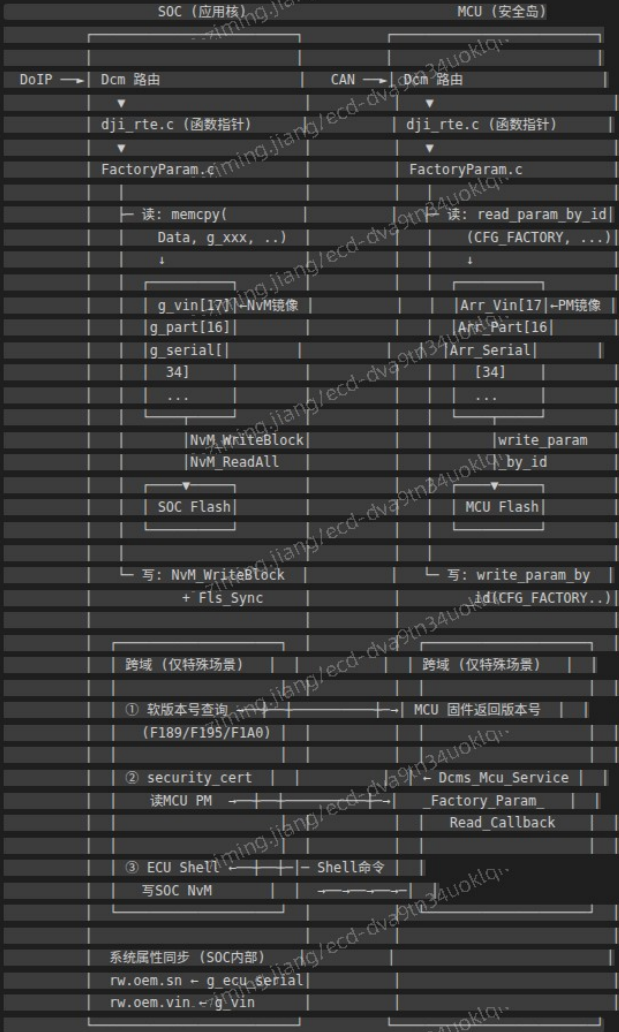
MCU 侧 read_param_by_id(CFG_FACTORY, ..., arr_vin) 也不是每次都读 Flash。PM 层的 Arr_Vin[17] (FactoryParam_Cfg.c.12) 同样是 PM block 的 RAM 镜像:


```
// FactoryParam_Cfg.c
uint8_t Arr_Vin[ARR_VIN_SIZE] = {0}; // PM RAM 镜像

// PM 参数注册
DEC_ARR("arr_vin", ..., pm_factory_param_block_t, arr_vin, ..., Arr_Vin, ...)

read_param_by_id() 从 Arr_Vin[] 读, write_param_by_id() 写 Arr_Vin[] 并触发 PM-NVM 异步写入 Flash。
```

三、一张图总结：工厂参数的存储与跨域



一句话总结:

- UDS DID 读写路径: 无跨域, 各自操作本地 Nvm/PM RAM 镜像
- RAM 镜像读: memcpy(Data, g_vin, 17) 读的就是当前数据, g_vin 在上电时已从 Flash 加载并被 Nvm 框架持续维护
- 跨域仅三种场景: 软件版本号查询 (SOC → MCU)、安全认证模块读 MCU PM (SOC → MCU)、ECU Shell 调试写 SOC NVM (MCU → SOC)